i would like to update my camera logic in my opengl scene so that it can be moved around a bit, but i want to also be restrictive about it, for example i want the user to be able to use the mouse to look around a bit, but rubber-banded to keep the focus on the central model, then id also like for the a,d/left,right keys to allow the user to circulate around the main model coordinates if possible, heres my scenebasic_uniform.cpp file, is there any more youd need to assist with this?

```
#include "scenebasic_uniform.h"

#include <cstdio>
#include <cstdlib>

#include <string>
using std::string;

#include <sstream>
#include <iostream>
using std::cerr;
using std::endl;

#include <GLFW/glfw3.h>
#include "helper/glutils.h"
#include "helper/texture.h"

using glm::vec3;
using glm::vec4;
using glm::mat3;
using glm::mat4;

SceneBasic_Uniform::SceneBasic_Uniform() : tPrev(0), angle(0.0f),
rotSpeed(glm::pi<float>() / 2.0f), plane(79.4f, 53.2f, 100, 100),
sky(100.0f) {
    LightsaberMesh = ObjMesh::load("media/Lightsaber_03.obj", true);
    BladeMEsh = ObjMesh::load("media/cylinder.obj", true);
}

void SceneBasic_Uniform::initScene()
{
    compile();
    glGenVertexArrays(1, &fsTriVao);

    prog.use();
    glEnable(GL_DEPTH_TEST);

    glEnable(GL_STENCIL_TEST);
    glClearStencil(0);

    model = mat4(1.0f);
    view = glm::lookAt(vec3(5.0f, 7.5f, 7.5f), vec3(0.0f, 0.75f, 0.0f),
vec3(0.0f, 1.0f, 0.0f));
    //view = glm::lookAt(camPos, camPos + camFront, camUp);

    GLFWwindow* win = glfwGetCurrentContext();
    if (win)
    {
        // Capture cursor
        glfwSetInputMode(win, GLFW_CURSOR, cursorCaptured ?
GLFW_CURSOR_DISABLED : GLFW_CURSOR_NORMAL);

        glfwGetCursorPos(win, &lastMouseX, &lastMouseY);
        firstMouse = true;
    }

    projection = mat4(1.0f);
    rotateModel = mat4(1.0f);
    rotateModel = glm::translate(rotateModel, vec3(4.0f, 0.26f, 5.0f));
    angle = 0.0f;


    //blinnPhongSpot stuff
    prog.setUniform("Spot.L", vec3(0.7f));
    prog.setUniform("Spot.La", vec3(0.5f));
    prog.setUniform("Spot.Exponent", 25.0f);
    prog.setUniform("Spot.Cutoff", glm::radians(20.0f));

    prog.setUniform("Light.L", vec3(0.5f));
    prog.setUniform("Light.La", vec3(0.25f));

    //fog
    prog.setUniform("Fog.MinDistance", 10.0f);
    prog.setUniform("Fog.MaxDistance", 45.0f);
    prog.setUniform("Fog.Colour", fogColour);
```

```
    cubeTex =
Texture::loadCubeMap("media/texture/cube/space/space");
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);


    LSdiffuseTexture =
Texture::loadTexture("media/texture/Lightsaber_03_exp_lambert1_BaseC
olor1.png");
    LSnormalMap =
Texture::loadTexture("media/texture/Lightsaber_03_exp_lambert1_Norm
al.png");
    LSmixingTexture = Texture::loadTexture("media/texture/rust.png");

    WorkbenchDiffuseMap =
Texture::loadTexture("media/texture/workbench.png");

    skyboxShader.use();
    skyboxShader.setUniform("FogColour", fogColour);

    bladeEmissive.use();
    bladeEmissive.setUniform("BladeColour",
bladeColours[bladeColourIndex]);
    bladeEmissive.setUniform("Intensity", 8.0f);
    bladeEmissive.setUniform("RimPower", 2.5f);
    bladeEmissive.setUniform("RimIntensity", 0.6f);

    fogColour = bladeColours[bladeColourIndex];
    prog.use();
    prog.setUniform("Fog.Colour", fogColour);

    updateFogColour();
}

void SceneBasic_Uniform::compile()
{
    try {
        prog.compileShader("shader/basic_uniform.vert");
        prog.compileShader("shader/basic_uniform.frag");
        prog.link();
        prog.use();

        bladeEmissive.compileShader("shader/blade.vert");
        bladeEmissive.compileShader("shader/blade.frag");
        bladeEmissive.link();
        bladeEmissive.use();

        skyboxShader.compileShader("shader/skybox.vert");
        skyboxShader.compileShader("shader/skybox.frag");
        skyboxShader.link();
        skyboxShader.use();

        hiltMaskProg.compileShader("shader/hilt_mask.vert");
        hiltMaskProg.compileShader("shader/hilt_mask.frag");
        hiltMaskProg.link();
        hiltMaskProg.use();

        edgeProg.compileShader("shader/edge_tri.vert");
        edgeProg.compileShader("shader/edge_detect.frag");
        edgeProg.link();
        edgeProg.use();
    }
    catch (GLSLProgramException& e) {
        cerr << e.what() << endl;
        exit(EXIT_FAILURE);
    }
}

void SceneBasic_Uniform::update(float t)
{
    float deltaT = t - tPrev;
    if (tPrev == 0.0f)
        deltaT = 0.0f;
    tPrev = t;
    angle += 0.25f * deltaT;
    if (angle > glm::two_pi<float>())
        angle -= glm::two_pi<float>();

    GLFWwindow* win = glfwGetCurrentContext();
    if (win)
    {
        //blade toggle
        if (glfwGetKey(win, GLFW_KEY_Q) == GLFW_PRESS &&
!Q_Pressed) {
            bladeOn = !bladeOn;
```

```
            updateFogColour();

        }
        Q_Pressed = (glfwGetKey(win, GLFW_KEY_Q) == GLFW_PRESS);
        // colour cycling
        if (glfwGetKey(win, GLFW_KEY_E) == GLFW_PRESS && !E_Pressed)
        {
            bladeColourIndex = (bladeColourIndex + 1) %
(int)bladeColours.size();

            bladeEmissive.use();
            bladeEmissive.setUniform("BladeColour",
bladeColours[bladeColourIndex]);

            updateFogColour();
        }
        E_Pressed = (glfwGetKey(win, GLFW_KEY_E) == GLFW_PRESS);
        //fog stuff
        if (glfwGetKey(win, GLFW_KEY_F) == GLFW_PRESS && !F_Pressed)
        {
            fogEnabled = !fogEnabled;
        }
        F_Pressed = (glfwGetKey(win, GLFW_KEY_F) == GLFW_PRESS);
        //rusty toggle
        if (glfwGetKey(win, GLFW_KEY_R) == GLFW_PRESS && !R_Pressed)
        {
            rusty = !rusty;
        }
        R_Pressed = (glfwGetKey(win, GLFW_KEY_R) == GLFW_PRESS);
        //rotate hilt
        if (glfwGetKey(win, GLFW_KEY_W) == GLFW_PRESS)
            hiltYawDeg -= hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_S) == GLFW_PRESS)
            hiltYawDeg += hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_UP) == GLFW_PRESS)
            hiltYawDeg -= hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_DOWN) == GLFW_PRESS)
            hiltYawDeg += hiltYawSpeed * deltaT;

        if (hiltYawDeg > 360.0f) hiltYawDeg -= 360.0f;
        if (hiltYawDeg < 0.0f)   hiltYawDeg += 360.0f;
    }
    render();

}

void SceneBasic_Uniform::render()
{

    prog.use();

    //glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT |
GL_STENCIL_BUFFER_BIT);

    // BlinnPhong spotlight
    glm::vec4 lightPosWorld = glm::vec4(10.0f * cos(angle), 10.0f, 10.0f *
sin(angle), 1.0f);
    prog.setUniform("Spot.Position", glm::vec3(view * lightPosWorld));

    glm::vec3 targetWorld = glm::vec3(0.0f, 2.0f, 3.0f);
    glm::vec3 lightDirWorldSpot = glm::normalize(targetWorld -
glm::vec3(lightPosWorld));

    glm::vec3 lightDirView = glm::mat3(view) * lightDirWorldSpot;
    prog.setUniform("Spot.Direction", lightDirView);


    glm::vec3 lightDirWorld = glm::normalize(glm::vec3(-0.6f, -1.0f,
-0.4f));
    glm::vec3 lightDirViewSpace = glm::mat3(view) * -lightDirWorld;
    prog.setUniform("Light.Position", glm::vec4(lightDirViewSpace, 0.0f));


    prog.setUniform("FogEnabled", fogEnabled ? 1 : 0);
    prog.setUniform("FogScale", fogScale);

    // Skybox stuff
    skyboxShader.use();
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);
    skyboxShader.setUniform("SkyboxTex", 0);

    glm::mat4 skyView = glm::mat4(glm::mat3(view));
    glm::mat4 skyMVP = projection * skyView;
    skyboxShader.setUniform("MVP", skyMVP);
```

```
    skyboxShader.setUniform("FogEnabled", fogEnabled ? 1 : 0);
    skyboxShader.setUniform("FogColour", fogColour);
    skyboxShader.setUniform("SkyFogAmount", skyFogAmount);

    sky.render();




    prog.use();
    //// Workbench
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, WorkbenchDiffuseMap);

    glActiveTexture(GL_TEXTURE2);
    glBindTexture(GL_TEXTURE_2D, WorkbenchDiffuseMap);

    prog.setUniform("Material.Ks", vec3(0.0f));
    prog.setUniform("Material.Ka", vec3(0.1f));
    prog.setUniform("Material.Shininess", 180.0f);

    model = mat4(1.0f);
    model = glm::translate(model, vec3(0.0f, -1.0f, 0.0f));
    model = glm::rotate(model, glm::radians(25.0f), vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, vec3(0.5f));
    setMatrices();
    plane.render();

    // Lightsaber hilt
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, LSdiffuseTexture);

    glActiveTexture(GL_TEXTURE1);
    glBindTexture(GL_TEXTURE_2D, LSnormalMap);

    if (rusty) {
        glActiveTexture(GL_TEXTURE2);
        glBindTexture(GL_TEXTURE_2D, LSmixingTexture);
    }
    else {
        glActiveTexture(GL_TEXTURE2);
        glBindTexture(GL_TEXTURE_2D, LSdiffuseTexture);
    }

    prog.setUniform("Material.Ks", vec3(0.25f));
    prog.setUniform("Material.Ka", vec3(0.15f));
    prog.setUniform("Material.Shininess", 80.0f);
    prog.setUniform("MixAmount", 1.0f);

    model = mat4(1.0f);
    model = glm::translate(model, vec3(-1.0f, 2.0f, 2.0f));
    model = glm::scale(model, vec3(1.5f));
    model = glm::rotate(model, glm::radians(270.0f), vec3(1.0f, 0.0f, 0.0f));
    model = glm::rotate(model, glm::radians(300.0f), vec3(0.0f, 0.0f, 1.0f));
    model = glm::rotate(model, glm::radians(hiltYawDeg), vec3(0.0f, 1.0f,
0.0f));

    setMatrices();
    LightsaberMesh->render();


    if (edgeEnabled)
    {
        // ---------- HILT EDGE PASS A: render hilt mask into FBO ----------
        glBindFramebuffer(GL_FRAMEBUFFER, hiltFbo);
        glViewport(0, 0, width, height);

        glDisable(GL_BLEND);
        glEnable(GL_DEPTH_TEST);
        glDepthMask(GL_TRUE);

        glClearColor(0, 0, 0, 0);
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

        hiltMaskProg.use();

        // Reuse the same model transform you used for the hilt
        // (IMPORTANT: copy/paste the same model matrix setup)
        model = mat4(1.0f);
        model = glm::translate(model, vec3(-1.0f, 2.0f, 2.0f));
        model = glm::scale(model, vec3(1.5f));
        model = glm::rotate(model, glm::radians(270.0f), vec3(1.0f, 0.0f,
0.0f));
        model = glm::rotate(model, glm::radians(300.0f), vec3(0.0f, 0.0f,
1.0f));
        model = glm::rotate(model, glm::radians(hiltYawDeg), vec3(0.0f,
1.0f, 0.0f));
```

```
        // For the mask shader we only need MVP
        mat4 mv = view * model;
        hiltMaskProg.setUniform("MVP", projection * mv);

        LightsaberMesh->render();

        // ---------- HILT EDGE PASS B: Sobel detect over main framebuffer
----------
        glBindFramebuffer(GL_FRAMEBUFFER, 0);
        glViewport(0, 0, width, height);

        glDisable(GL_DEPTH_TEST);
        glDepthMask(GL_FALSE);

        glEnable(GL_BLEND);
        glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

        edgeProg.use();
        glActiveTexture(GL_TEXTURE0);
        glBindTexture(GL_TEXTURE_2D, hiltMaskTex);
        edgeProg.setUniform("MaskTex", 0);

        edgeProg.setUniform("TexelSize", glm::vec2(1.0f / width, 1.0f /
height));
        edgeProg.setUniform("Threshold", edgeThreshold);
        edgeProg.setUniform("Opacity", edgeOpacity);
        edgeProg.setUniform("Thickness", edgeThickness);
        edgeProg.setUniform("EdgeColour", edgeColour);

        glBindVertexArray(fsTriVao);
        glDrawArrays(GL_TRIANGLES, 0, 3);
        glBindVertexArray(0);

        // Restore defaults
        glDisable(GL_BLEND);
        glDepthMask(GL_TRUE);
        glEnable(GL_DEPTH_TEST);
    }


    // Blade
    if (bladeOn)
    {
        glm::mat4 bladeBase = mat4(1.0f);
        bladeBase = glm::translate(bladeBase, vec3(10.0f, 2.1f, -4.0f));
        bladeBase = glm::rotate(bladeBase, glm::radians(270.0f), vec3(1.0f,
0.0f, 0.0f));
        bladeBase = glm::rotate(bladeBase, glm::radians(300.0f), vec3(0.0f,
0.0f, 1.0f));

        // -------- Pass 1: White core (writes stencil = 1 where the blade is)
--------
        bladeEmissive.use();

        glDisable(GL_BLEND);
        glEnable(GL_DEPTH_TEST);
        glDepthMask(GL_TRUE);

        // Stencil: always write 1 where fragments pass depth test
        glStencilMask(0xFF);
        glStencilFunc(GL_ALWAYS, 1, 0xFF);
        glStencilOp(GL_KEEP, GL_KEEP, GL_REPLACE);

        model = bladeBase;
        model = glm::scale(model, vec3(0.4f));
        setMatricesEmiss();

        bladeEmissive.setUniform("BladeColour", glm::vec3(1.0f));
        bladeEmissive.setUniform("Intensity", 10.0f);
        bladeEmissive.setUniform("RimPower", 5.0f);
        bladeEmissive.setUniform("RimIntensity", 0.25f);

        bladeEmissive.setUniform("Alpha", 1.0f);
        bladeEmissive.setUniform("CoreAlpha", 1.0f);

        BladeMEsh->render();

        // -------- Pass 2: Coloured glow shell (only OUTSIDE the core
silhouette) --------
        glEnable(GL_BLEND);
        glBlendFunc(GL_ONE, GL_ONE);

        // Don't write depth for glow, but DO depth test so hilt blocks it
        glDepthMask(GL_FALSE);

        // Stencil: only draw where stencil != 1 (i.e., outside the core)
```

```
        glStencilMask(0x00);
        glStencilFunc(GL_NOTEQUAL, 1, 0xFF);
        glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);

        // Optional but usually improves the “wrap” look:
        // render only backfaces of the enlarged shell so it appears as an
outline
        glEnable(GL_CULL_FACE);
        glCullFace(GL_FRONT);

        model = bladeBase;
        model = glm::scale(model, vec3(0.4f * 1.4f));
        setMatricesEmiss();

        bladeEmissive.setUniform("BladeColour",
bladeColours[bladeColourIndex]);
        bladeEmissive.setUniform("Intensity", 2.5f);
        bladeEmissive.setUniform("RimPower", 2.0f);
        bladeEmissive.setUniform("RimIntensity", 1.0f);

        bladeEmissive.setUniform("Alpha", 0.55f);
        bladeEmissive.setUniform("CoreAlpha", 0.0f);

        BladeMEsh->render();

        // -------- Restore state --------
        glCullFace(GL_BACK);
        glDisable(GL_CULL_FACE);

        glDepthMask(GL_TRUE);
        glDisable(GL_BLEND);

        glStencilMask(0xFF);
        glStencilFunc(GL_ALWAYS, 0, 0xFF);
        glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);
    }
}

void SceneBasic_Uniform::setMatrices() {
    mat4 mv = view * model;
    prog.setUniform("ModelViewMatrix", mv);
    prog.setUniform("NormalMatrix", glm::mat3(vec3(mv[0]), vec3(mv[1]),
vec3(mv[2])));
    prog.setUniform("MVP", projection * mv);
}

void SceneBasic_Uniform::setMatricesEmiss() {
    mat4 mv = view * model;
    bladeEmissive.setUniform("ModelViewMatrix", mv);
    bladeEmissive.setUniform("NormalMatrix", glm::mat3(vec3(mv[0]),
vec3(mv[1]), vec3(mv[2])));
    bladeEmissive.setUniform("MVP", projection * mv);
}

void SceneBasic_Uniform::updateFogColour()
{
    fogColour = bladeOn ? bladeColours[bladeColourIndex] : fogGrey;

    // Update main scene shader fog colour
    prog.use();
    prog.setUniform("Fog.Colour", fogColour);

    // Update skybox fog colour too (so it stays consistent)
    skyboxShader.use();
    skyboxShader.setUniform("FogColour", fogColour);
}

void SceneBasic_Uniform::resize(int w, int h) {
    glViewport(0, 0, w, h);
    width = w;
    height = h;
    projection = glm::perspective(glm::radians(70.0f), (float)w / h, 0.3f,
100.0f);
    initHiltEdgeFbo(w, h);
}

void SceneBasic_Uniform::initHiltEdgeFbo(int w, int h)
{
    // Clean up old
    if (hiltMaskTex) glDeleteTextures(1, &hiltMaskTex);
    if (hiltDepthRb) glDeleteRenderbuffers(1, &hiltDepthRb);
    if (hiltFbo) glDeleteFramebuffers(1, &hiltFbo);

    // Colour mask texture
    glGenTextures(1, &hiltMaskTex);
    glBindTexture(GL_TEXTURE_2D, hiltMaskTex);
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA8, w, h, 0, GL_RGBA,
```

```
GL_UNSIGNED_BYTE, nullptr);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
GL_CLAMP_TO_EDGE);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
GL_CLAMP_TO_EDGE);

    // Depth buffer
    glGenRenderbuffers(1, &hiltDepthRb);
    glBindRenderbuffer(GL_RENDERBUFFER, hiltDepthRb);
    glRenderbufferStorage(GL_RENDERBUFFER,
GL_DEPTH_COMPONENT24, w, h);

    // FBO
    glGenFramebuffers(1, &hiltFbo);
    glBindFramebuffer(GL_FRAMEBUFFER, hiltFbo);
    glFramebufferTexture2D(GL_FRAMEBUFFER,
GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, hiltMaskTex, 0);
    glFramebufferRenderbuffer(GL_FRAMEBUFFER,
GL_DEPTH_ATTACHMENT, GL_RENDERBUFFER, hiltDepthRb);

    GLenum drawBufs[] = { GL_COLOR_ATTACHMENT0 };
    glDrawBuffers(1, drawBufs);

    GLenum status = glCheckFramebufferStatus(GL_FRAMEBUFFER);
    if (status != GL_FRAMEBUFFER_COMPLETE) {
        std::cerr << "Hilt edge FBO incomplete: " << status << std::endl;
    }

    glBindFramebuffer(GL_FRAMEBUFFER, 0);
}
```

Thought for 6s

You've already given basically everything I need from the `.cpp` side. The only extra bit that matters is what you currently have in `scenebasic_uniform.h` (just so I don't clash with your existing camera vars like `cursorCaptured`, `lastMouseX`, etc.). But even without it, we can drop in an orbit-style camera cleanly.

What you're describing is an **orbit camera**:

- Camera always looks at a **fixed target** (your central model / "focus point").
- Mouse *nudges* the orbit (yaw/pitch), but is **clamped** so you can't look away too far ("rubber band").
- `A/D` (and left/right) add a steady yaw orbit around the target.

Below is a solid, restrictive implementation that will work with your current structure (GLFW polling in `update()`).

---

1) Pick a focus point (target) and a default orbit

Right now your view is:

```
C++
```

So your "central focus" is `vec3(0.0f, 0.75f, 0.0f)`.

Let's formalise that as:

```
glm::vec3 camTarget = glm::vec3(0.0f, 0.75f, 0.0f);
float camDistance = 11.0f;  // tweak
C++
```

---

2) Add these members to `SceneBasic_Uniform` (in your header)

Add something like this (names won't matter as long as you use them consistently):

```
// Orbit camera
glm::vec3 camTarget = glm::vec3(0.0f, 0.75f, 0.0f);
float camDistance = 11.0f;

float camYawDeg = -45.0f;
float camPitchDeg = -20.0f;

float camYawSpeedDeg = 90.0f;    // A/D orbit speed
float mouseSensitivity = 0.08f;  // mouse influence

// "Rubber band" clamp limits (restrict look)
```

```cpp
float pitchMinDeg = -35.0f;
float pitchMaxDeg =  10.0f;

// Optional: keep yaw within a limited window too (rubber band feel)
bool limitYaw = false;
float yawCentreDeg = -45.0f;
float yawLimitDeg = 35.0f; // +/- from centre
```

C++

```cpp
double lastMouseX = 0.0, lastMouseY = 0.0;
bool firstMouse = true;
bool cursorCaptured = true;
```

If you **already** have `lastMouseX/lastMouseY/firstMouse/cursorCaptured`, don't duplicate them — just reuse yours.

---

3) Create an `updateCamera(deltaT)` and call it in `update()`

Call site (in your `update(float t)`)

Right before `render();`, do:

C++

Implementation

Drop this into your `.cpp`:

```cpp
static float wrapDeg(float a)
{
    while (a > 180.0f) a -= 360.0f;
    while (a < -180.0f) a += 360.0f;
    return a;
}

void SceneBasic_Uniform::updateCamera(float dt)
{
    GLFWwindow* win = glfwGetCurrentContext();
    if (!win) return;

    // --- Keyboard orbit (A/D or Left/Right) ---
    float yawInput = 0.0f;
    if (glfwGetKey(win, GLFW_KEY_A) == GLFW_PRESS || glfwGetKey(win, GLFW_KEY_LEFT) == GLFW_PRESS
        yawInput -= 1.0f;
    if (glfwGetKey(win, GLFW_KEY_D) == GLFW_PRESS || glfwGetKey(win, GLFW_KEY_RIGHT) == GLFW_PRES
        yawInput += 1.0f;

    camYawDeg += yawInput * camYawSpeedDeg * dt;

    // --- Mouse orbit (only if cursor captured) ---
    if (cursorCaptured)
    {
        double mx, my;
        glfwGetCursorPos(win, &mx, &my);

        if (firstMouse)
        {
            lastMouseX = mx;
            lastMouseY = my;
            firstMouse = false;
        }

        double dx = mx - lastMouseX;
        double dy = my - lastMouseY;
        lastMouseX = mx;
        lastMouseY = my;

        camYawDeg   += (float)dx * mouseSensitivity;
        camPitchDeg -= (float)dy * mouseSensitivity; // invert so moving mouse up looks down less

        // Pitch clamp = "rubber band" vertical restriction
        camPitchDeg = glm::clamp(camPitchDeg, pitchMinDeg, pitchMaxDeg);

        // Optional yaw clamp window = "rubber band" horizontal restriction
        if (limitYaw)
        {
            float rel = wrapDeg(camYawDeg - yawCentreDeg);
            rel = glm::clamp(rel, -yawLimitDeg, yawLimitDeg);
            camYawDeg = yawCentreDeg + rel;
        }
    }

    // --- Build orbit position from yaw/pitch/distance ---
    float yawRad   = glm::radians(camYawDeg);
    float pitchRad = glm::radians(camPitchDeg);

    // Spherical to Cartesian (Y up)
    glm::vec3 offset;
    offset.x = camDistance * cosf(pitchRad) * cosf(yawRad);
```

C++

```cpp
    glm::vec3 camPos = camTarget + offset;

    // Always look at the target (this is what keeps focus locked)
```

```
    view = glm::lookAt(camPos, camTarget, glm::vec3(0,1,0));
}
```

This gives you:

- Mouse nudges yaw/pitch, but pitch is clamped so you can't swing over/under wildly.
- A/D (or left/right) steadily orbits around the same target.
- You always look at the target, so you never "lose" the central model.

---

4) Make it feel "rubber-banded" (tighter, nicer)

The clamping already *is* the restriction, but if you want it to feel more like it "springs back" toward centre instead of hard-stopping, add a gentle recenter when there's no input:

Add members:

```cpp
float yawReturnSpeed = 3.0f;   // higher = snappier return
float pitchReturnSpeed = 3.0f;
```

Then at the end of the mouse section (or after input), do:

```cpp
if (autoRecentre)
{
    // If no orbit keys pressed, drift back toward centre yaw
    if (yawInput == 0.0f)
        camYawDeg = glm::mix(camYawDeg, yawCentreDeg, 1.0f - expf(-yawReturnSpeed * dt));
```

That gives the "rubber band" feel instead of a rigid clamp.

---

5) One important gotcha in *your* render code

You compute spotlight position in **view space**:

```cpp
```

So once the camera moves, the light behaviour may *appear* to shift (because you're explicitly converting to view space every frame — which is correct if your shader expects view-space). Just be aware: moving the camera will change those uniforms, so if anything looks "odd", it's probably just that your lighting is view-space dependent (not wrong, just noticeable).

Is there anything else I'd need?

Only if:

- your header already has camera vars (so we don't duplicate / conflict), or
- you want the orbit target to follow your saber position instead of being fixed.

If you paste your `scenebasic_uniform.h` (just the member variables part), I can splice this in *exactly* to match what you already have, no guesswork.

---

okay so before i use any of that code, heres my current scenebasic_uniform.h

```
#ifndef SCENEBASIC_UNIFORM_H
#define SCENEBASIC_UNIFORM_H

#include "helper/scene.h"

#include <glm/glm.hpp>
#include <glad/glad.h>
#include "helper/glslprogram.h"
#include "helper/torus.h"
#include "helper/teapot.h"
#include "helper/plane.h"
#include "helper/objmesh.h"
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <array>
#include "helper/cube.h"
#include "helper/skybox.h"
```

```
class SceneBasic_Uniform : public Scene
{
private:
    glm::mat4 rotationMatrix;
    SkyBox sky;
    Plane plane;
    Cube cube;
    //std::unique_ptr<ObjMesh> C1Mesh;
    std::unique_ptr<ObjMesh> LightsaberMesh;
    std::unique_ptr<ObjMesh> BladeMEsh;
    glm::mat4 rotateModel;
    //GLuint C1diffuseTexture, C1normalMap;
    GLuint LSdiffuseTexture, LSnormalMap, LSmixingTexture;
    //GLuint BladeDiffuseTexture;
    GLuint WorkbenchDiffuseMap;
    GLuint cubeTex;


    float tPrev = 0.0f;
    float angle = 0.0f;
    float rotSpeed;
    bool bladeOn = false;
    bool fogEnabled = true;
    float fogScale = 1.0f;
    bool rusty = false;

    bool Q_Pressed = false;
    bool E_Pressed = false;
    bool F_Pressed = false;
    bool R_Pressed = false;

    glm::vec3 bladeColour;
    std::array<glm::vec3, 6> bladeColours = {
        glm::vec3(0.0f, 0.5f, 1.0f),  // BLUE
        glm::vec3(0.0f, 1.0f, 0.0f),   // GREEN
        glm::vec3(1.0f, 1.0f, 0.0f),   // YELLOW
        glm::vec3(1.0f, 0.0f, 0.0f),   // RED
        glm::vec3(1.0f, 0.0f, 1.0f),   // MAGENTA
        glm::vec3(1.0f, 1.0f, 1.0f),    // WHITE
    };
    int bladeColourIndex = 0;
    glm::vec3 fogColour = glm::vec3(0.6f, 0.6f, 0.6f);
    const glm::vec3 fogGrey = glm::vec3(0.6f, 0.6f, 0.6f);
    float skyFogAmount = 0.35f;


    GLSLProgram prog, skyboxShader, bladeEmissive;
    void compile();
    void setMatrices();
    void setMatricesEmiss();
    void updateFogColour();

    // Camera stuff
    glm::vec3 camPos = glm::vec3(5.0f, 7.5f, 7.5f);
    glm::vec3 camFront = glm::normalize(glm::vec3(0.0f, 0.75f, 0.0f) -
camPos);
    glm::vec3 camUp = glm::vec3(0.0f, 1.0f, 0.0f);

    bool firstMouse = true;
    double lastMouseX = 0.0;
    double lastMouseY = 0.0;


    float hiltYawDeg = 0.0f;
    float hiltYawSpeed = 90.0f;
    bool cursorCaptured = true;
    bool escWasDown = false;

    // --- Edge detect (hilt only) ---
    GLSLProgram hiltMaskProg;
    GLSLProgram edgeProg;

    GLuint hiltFbo = 0;
    GLuint hiltMaskTex = 0;
    GLuint hiltDepthRb = 0;

    GLuint fsTriVao = 0;   // fullscreen triangle VAO
    bool edgeEnabled = true; // optional toggle if you want

    // tweakables
    float edgeThreshold = 0.2f;   // higher = fewer edges
    float edgeOpacity = 1.0f;   // 0..1
    float edgeThickness = 1.0f;   // 1 = normal, 2 = thicker
    glm::vec3 edgeColour = glm::vec3(1.0f); // black outline
    void initHiltEdgeFbo(int w, int h);
```

```
public:
    SceneBasic_Uniform();

    void initScene();
    void update( float t );
    void render();
    void resize(int, int);

};

#endif // SCENEBASIC_UNIFORM_H
```

Nice — this header is plenty, and it actually makes this cleaner because you've already got `camPos`, `camFront`, `firstMouse`, `lastMouseX/Y`, `cursorCaptured`, etc.

Right now your camera members are **FPS-style** (`camPos + camFront`). What you want is an **orbit camera**, so we'll replace the meaning of `camFront` (you won't really need it), and instead drive `camPos` from yaw/pitch/distance around a fixed `camTarget`.

Below is a minimal, *restrictive* orbit setup that fits your existing variables and style.

---

1) Update `scenebasic_uniform.h` camera section

Replace your current "Camera stuff" block with this (keep your existing `firstMouse/lastMouseX/lastMouseY/cursorCaptured/escWasDown` as they already exist — we'll reuse them):

```cpp
// Camera stuff (ORBIT)
glm::vec3 camTarget = glm::vec3(0.0f, 0.75f, 0.0f); // focus point (central model)
float camDistance = 11.0f;

// Orbit angles
float camYawDeg   = -45.0f;
float camPitchDeg = -20.0f;

// Speeds / feel
float camYawSpeedDeg = 90.0f;     // A/D orbit speed
float mouseSensitivity = 0.08f;   // mouse orbit influence

// Restriction ("rubber band" limits)
float pitchMinDeg = -35.0f;
float pitchMaxDeg =  10.0f;

// Optional horizontal restriction
bool  limitYaw = false;
float yawCentreDeg = -45.0f;
float yawLimitDeg  = 35.0f;       // +/- from centre

// Derived / stored camera position
glm::vec3 camPos = glm::vec3(5.0f, 7.5f, 7.5f);
glm::vec3 camUp  = glm::vec3(0.0f, 1.0f, 0.0f);

// Keep these (you already have them below)
```

C++

```cpp
double lastMouseY = 0.0;
bool cursorCaptured = true;
bool escWasDown = false;
```

✅ Notice: `camFront` is gone (orbit camera doesn't really use it). If your code still references `camFront` anywhere else, delete/replace those references.

---

2) Add a function declaration in the header

In `private:` add:

C++

(Anywhere near `setMatrices()` etc is fine.)

---

3) In `initScene()`, set the initial orbit from your current view

Right now you hardcode:

C++

Replace it with:

```cpp
camPos = glm::vec3(5.0f, 7.5f, 7.5f);
camTarget = glm::vec3(0.0f, 0.75f, 0.0f);
```

```
// If you want: derive distance from those two points so it matches exactly
camDistance = glm::length(camPos - camTarget);

// Keep your preferred starting angles (or tweak)
camYawDeg = -45.0f;
```

C++

```
updateCamera(0.0f); // builds view from orbit values
```

That way the view matrix is always controlled by the orbit system, not a one-off lookAt.

---

4) Call camera update each frame (in `update(float t)`)

Near the end of your `update()` (before `render()`):

C++

---

5) Add `updateCamera(dt)` implementation to your `.cpp`

Drop this into `scenebasic_uniform.cpp` (below `compile()` is a good spot):

```
static float wrapDeg(float a)
{
    while (a > 180.0f) a -= 360.0f;
    while (a < -180.0f) a += 360.0f;
    return a;
}

void SceneBasic_Uniform::updateCamera(float dt)
{
    GLFWwindow* win = glfwGetCurrentContext();
    if (!win) return;

    // Orbit with A/D (and Left/Right)
    float yawInput = 0.0f;
    if (glfwGetKey(win, GLFW_KEY_A) == GLFW_PRESS || glfwGetKey(win, GLFW_KEY_LEFT) == GLFW_PRESS
        yawInput -= 1.0f;
    if (glfwGetKey(win, GLFW_KEY_D) == GLFW_PRESS || glfwGetKey(win, GLFW_KEY_RIGHT) == GLFW_PRES
        yawInput += 1.0f;

    camYawDeg += yawInput * camYawSpeedDeg * dt;

    // Mouse orbit (only while cursor is captured)
    if (cursorCaptured)
    {
        double mx, my;
        glfwGetCursorPos(win, &mx, &my);

        if (firstMouse)
        {
            lastMouseX = mx;
            lastMouseY = my;
            firstMouse = false;
        }

        double dx = mx - lastMouseX;
        double dy = my - lastMouseY;
        lastMouseX = mx;
        lastMouseY = my;

        camYawDeg   += (float)dx * mouseSensitivity;
        camPitchDeg -= (float)dy * mouseSensitivity; // move mouse up -> pitch up

        // Vertical restriction
        camPitchDeg = glm::clamp(camPitchDeg, pitchMinDeg, pitchMaxDeg);

        // Optional horizontal restriction
        if (limitYaw)
        {
            float rel = wrapDeg(camYawDeg - yawCentreDeg);
            rel = glm::clamp(rel, -yawLimitDeg, yawLimitDeg);
            camYawDeg = yawCentreDeg + rel;
        }
    }

    // Build orbit position from yaw/pitch/distance
    float yawRad   = glm::radians(camYawDeg);
    float pitchRad = glm::radians(camPitchDeg);

    glm::vec3 offset;
    offset.x = camDistance * cosf(pitchRad) * cosf(yawRad);
```

C++

```
    camPos = camTarget + offset;

    // Always keep focus on the target (hard lock)
    view = glm::lookAt(camPos, camTarget, camUp);
}
```

---

6) Rubber-band feel (optional but nice)

If you want it to *drift back* toward centre (instead of just clamping), add these members:

```cpp
bool autoRecentre = true;
float yawReturnSpeed = 3.0f;
```

Then inside `updateCamera`, after inputs, add:

```cpp
if (autoRecentre)
{
    if (yawInput == 0.0f)
        camYawDeg = glm::mix(camYawDeg, yawCentreDeg, 1.0f - expf(-yawReturnSpeed * dt));
```

This is the bit that makes it feel "rubber-banded" rather than a hard stop.

One small warning with your current code

You're updating spotlight uniforms in view space:

```cpp
```

That's fine, but once you move the camera you'll *notice* lighting shift relative to camera (because it's meant to). If you expected the light to be world-locked visually, that's where you'd adjust (but don't change it yet unless it becomes a problem).

If you paste just the part of your `.cpp` where you handle **ESC cursor capture** (you had it in an older snippet), I can merge that cleanly so mouse orbit only happens while captured, and the first mouse sample doesn't jump when you re-capture.

---

okay thats mostly good, although quite wonky comapred to what i had expected, i wanted the camera to stay at the height it was originally at, right now what this code does is it starts (and pushes) the camera down below the model which pushes it under my plane which just looks wrong.
i wanted it to stay roughly at the height that it currently has as its top bounds, how would we resolve this

What's happening is simply this: your **initial pitch was actually positive (camera above the target)**, but we hard-set `camPitchDeg = -20.0f`, which forces the orbit maths to put the camera *below* the target on start.

With your original values:

- `camPos = (5, 7.5, 7.5)`
- `camTarget = (0, 0.75, 0)`
- vertical offset is `+6.75`, so the pitch should be ~ **+37°**, not negative.

Fix: derive yaw/pitch from your existing starting camera position

Do this once in `initScene()` after you set `camPos` / `camTarget`:

```cpp
// Initialise orbit angles from your current camera placement
glm::vec3 off = camPos - camTarget;
camDistance = glm::length(off);

glm::vec3 dir = off / camDistance; // normalised
```

Then set your pitch limits *around that* so it never drops below where you want. For example, if you want "current height is the lowest it can go":

```cpp
// Keep the camera at/above the starting height angle
```

If you want a little bit of down movement but not enough to go under your plane:

```
C++
```

And **delete** any hard-coded `camYawDeg = -45` / `camPitchDeg = -20` you had before (those are what caused the "push down").

Optional: enforce a hard world-space height floor (extra safety)

If you *never* want the camera below a certain Y regardless of pitch, clamp it after computing `camPos` in `updateCamera()`:

```
float minCamY = 1.0f; // or whatever is safely above your plane
if (camPos.v < minCamY) camPos.v = minCamY;
C++
```

This is a bit less "pure orbit" (it squashes the sphere), but it's an effective guardrail.

If you apply just the **derived yaw/pitch in initScene** + **pitchMinDeg = camPitchDeg**, your camera will start exactly where it used to and will not drop below that height.

1/2

heres the .h:

```
#ifndef SCENEBASIC_UNIFORM_H
#define SCENEBASIC_UNIFORM_H

#include "helper/scene.h"

#include <glm/glm.hpp>
#include <glad/glad.h>
#include "helper/glslprogram.h"
#include "helper/torus.h"
#include "helper/teapot.h"
#include "helper/plane.h"
#include "helper/objmesh.h"
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <array>
#include "helper/cube.h"
#include "helper/skybox.h"


class SceneBasic_Uniform : public Scene
{
private:
    glm::mat4 rotationMatrix;
    SkyBox sky;
    Plane plane;
    Cube cube;
    //std::unique_ptr<ObjMesh> C1Mesh;
    std::unique_ptr<ObjMesh> LightsaberMesh;
    std::unique_ptr<ObjMesh> BladeMEsh;
    glm::mat4 rotateModel;
    //GLuint C1diffuseTexture, C1normalMap;
    GLuint LSdiffuseTexture, LSnormalMap, LSmixingTexture;
    //GLuint BladeDiffuseTexture;
    GLuint WorkbenchDiffuseMap;
    GLuint cubeTex;


    float tPrev = 0.0f;
    float angle = 0.0f;
    float rotSpeed;
    bool bladeOn = false;
    bool fogEnabled = true;
    float fogScale = 1.0f;
    bool rusty = false;

    bool Q_Pressed = false;
    bool E_Pressed = false;
    bool F_Pressed = false;
    bool R_Pressed = false;

    glm::vec3 bladeColour;
    std::array<glm::vec3, 6> bladeColours = {
        glm::vec3(0.0f, 0.5f, 1.0f),  // BLUE
        glm::vec3(0.0f, 1.0f, 0.0f),   // GREEN
```

```
        glm::vec3(1.0f, 1.0f, 0.0f),   // YELLOW
        glm::vec3(1.0f, 0.0f, 0.0f),   // RED
        glm::vec3(1.0f, 0.0f, 1.0f),   // MAGENTA
        glm::vec3(1.0f, 1.0f, 1.0f),    // WHITE
    };
    int bladeColourIndex = 0;
    glm::vec3 fogColour = glm::vec3(0.6f, 0.6f, 0.6f);
    const glm::vec3 fogGrey = glm::vec3(0.6f, 0.6f, 0.6f);
    float skyFogAmount = 0.35f;


    GLSLProgram prog, skyboxShader, bladeEmissive;
    void compile();
    void setMatrices();
    void setMatricesEmiss();
    void updateFogColour();
    void updateCamera(float dt);

    // Camera stuff
    ///glm::vec3 camPos = glm::vec3(5.0f, 7.5f, 7.5f);
    glm::vec3 camFront = glm::normalize(glm::vec3(0.0f, 0.75f, 0.0f) -
camPos);
    //glm::vec3 camUp = glm::vec3(0.0f, 1.0f, 0.0f);

    //bool firstMouse = true;
    //double lastMouseX = 0.0;
    //double lastMouseY = 0.0;


    float hiltYawDeg = 0.0f;
    float hiltYawSpeed = 90.0f;
    //bool cursorCaptured = true;
    //bool escWasDown = false;



    // Camera stuff (ORBIT)
    glm::vec3 camTarget = glm::vec3(0.0f, 0.75f, 0.0f); // focus point
(central model)
    float camDistance = 11.0f;

    // Orbit angles
    float camYawDeg = -45.0f;
    float camPitchDeg = 35.0f;

    // Speeds / feel
    float camYawSpeedDeg = 90.0f;     // A/D orbit speed
    float mouseSensitivity = 0.08f;   // mouse orbit influence

    // Restriction ("rubber band" limits)
    float pitchMinDeg = 15.0f;
    float pitchMaxDeg = 55.0f;

    // Optional horizontal restriction
    bool  limitYaw = false;
    float yawCentreDeg = -45.0f;
    float yawLimitDeg = 35.0f;       // +/- from centre

    // Derived / stored camera position
    glm::vec3 camPos = glm::vec3(5.0f, 7.5f, 7.5f);
    glm::vec3 camUp = glm::vec3(0.0f, 1.0f, 0.0f);

    // Keep these (you already have them below)
    bool firstMouse = true;
    double lastMouseX = 0.0;
    double lastMouseY = 0.0;
    bool cursorCaptured = true;
    bool escWasDown = false;

    // auto center
    bool autoRecentre = true;
    float yawReturnSpeed = 3.0f;
    float pitchReturnSpeed = 3.0f;
    float pitchCentreDeg = -20.0f;



    // --- Edge detect (hilt only) ---
    GLSLProgram hiltMaskProg;
    GLSLProgram edgeProg;

    GLuint hiltFbo = 0;
    GLuint hiltMaskTex = 0;
    GLuint hiltDepthRb = 0;

    GLuint fsTriVao = 0;   // fullscreen triangle VAO
    bool edgeEnabled = true; // optional toggle if you want
```

```
    // tweakables
    float edgeThreshold = 0.2f;   // higher = fewer edges
    float edgeOpacity = 1.0f;   // 0..1
    float edgeThickness = 1.0f;   // 1 = normal, 2 = thicker
    glm::vec3 edgeColour = glm::vec3(1.0f); // black outline
    void initHiltEdgeFbo(int w, int h);

public:
    SceneBasic_Uniform();

    void initScene();
    void update( float t );
    void render();
    void resize(int, int);

};

#endif // SCENEBASIC_UNIFORM_H

and the .cpp
#include "scenebasic_uniform.h"

#include <cstdio>
#include <cstdlib>

#include <string>
using std::string;

#include <sstream>
#include <iostream>
using std::cerr;
using std::endl;

#include <GLFW/glfw3.h>
#include "helper/glutils.h"
#include "helper/texture.h"

using glm::vec3;
using glm::vec4;
using glm::mat3;
using glm::mat4;

SceneBasic_Uniform::SceneBasic_Uniform() : tPrev(0), angle(0.0f),
rotSpeed(glm::pi<float>() / 2.0f), plane(79.4f, 53.2f, 100, 100),
sky(100.0f) {
    LightsaberMesh = ObjMesh::load("media/Lightsaber_03.obj", true);
    BladeMEsh = ObjMesh::load("media/cylinder.obj", true);
}

void SceneBasic_Uniform::initScene()
{
    compile();
    glGenVertexArrays(1, &fsTriVao);

    prog.use();
    glEnable(GL_DEPTH_TEST);

    glEnable(GL_STENCIL_TEST);
    glClearStencil(0);

    model = mat4(1.0f);
    //view = glm::lookAt(vec3(5.0f, 7.5f, 7.5f), vec3(0.0f, 0.75f, 0.0f),
vec3(0.0f, 1.0f, 0.0f));
    //view = glm::lookAt(camPos, camPos + camFront, camUp);
    camPos = glm::vec3(5.0f, 7.5f, 7.5f);
    camTarget = glm::vec3(0.0f, 0.75f, 0.0f);

    // If you want: derive distance from those two points so it matches
exactly
    camDistance = glm::length(camPos - camTarget);

    // Keep your preferred starting angles (or tweak)
    camYawDeg = -45.0f;
    camPitchDeg = -20.0f;

    updateCamera(0.0f); // builds view from orbit values

    GLFWwindow* win = glfwGetCurrentContext();
    if (win)
    {
        // Capture cursor
        glfwSetInputMode(win, GLFW_CURSOR, cursorCaptured ?
GLFW_CURSOR_DISABLED : GLFW_CURSOR_NORMAL);

        glfwGetCursorPos(win, &lastMouseX, &lastMouseY);
        firstMouse = true;
```

```
    }

    projection = mat4(1.0f);
    rotateModel = mat4(1.0f);
    rotateModel = glm::translate(rotateModel, vec3(4.0f, 0.26f, 5.0f));
    angle = 0.0f;


    //blinnPhongSpot stuff
    prog.setUniform("Spot.L", vec3(0.7f));
    prog.setUniform("Spot.La", vec3(0.5f));
    prog.setUniform("Spot.Exponent", 25.0f);
    prog.setUniform("Spot.Cutoff", glm::radians(20.0f));

    prog.setUniform("Light.L", vec3(0.5f));
    prog.setUniform("Light.La", vec3(0.25f));

    //fog
    prog.setUniform("Fog.MinDistance", 10.0f);
    prog.setUniform("Fog.MaxDistance", 45.0f);
    prog.setUniform("Fog.Colour", fogColour);


    cubeTex =
Texture::loadCubeMap("media/texture/cube/space/space");
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);


    LSdiffuseTexture =
Texture::loadTexture("media/texture/Lightsaber_03_exp_lambert1_BaseC
olor1.png");
    LSnormalMap =
Texture::loadTexture("media/texture/Lightsaber_03_exp_lambert1_Norm
al.png");
    LSmixingTexture = Texture::loadTexture("media/texture/rust.png");

    WorkbenchDiffuseMap =
Texture::loadTexture("media/texture/workbench.png");

    skyboxShader.use();
    skyboxShader.setUniform("FogColour", fogColour);

    bladeEmissive.use();
    bladeEmissive.setUniform("BladeColour",
bladeColours[bladeColourIndex]);
    bladeEmissive.setUniform("Intensity", 8.0f);
    bladeEmissive.setUniform("RimPower", 2.5f);
    bladeEmissive.setUniform("RimIntensity", 0.6f);

    fogColour = bladeColours[bladeColourIndex];
    prog.use();
    prog.setUniform("Fog.Colour", fogColour);

    updateFogColour();
}

void SceneBasic_Uniform::compile()
{
    try {
        prog.compileShader("shader/basic_uniform.vert");
        prog.compileShader("shader/basic_uniform.frag");
        prog.link();
        prog.use();

        bladeEmissive.compileShader("shader/blade.vert");
        bladeEmissive.compileShader("shader/blade.frag");
        bladeEmissive.link();
        bladeEmissive.use();

        skyboxShader.compileShader("shader/skybox.vert");
        skyboxShader.compileShader("shader/skybox.frag");
        skyboxShader.link();
        skyboxShader.use();

        hiltMaskProg.compileShader("shader/hilt_mask.vert");
        hiltMaskProg.compileShader("shader/hilt_mask.frag");
        hiltMaskProg.link();
        hiltMaskProg.use();

        edgeProg.compileShader("shader/edge_tri.vert");
        edgeProg.compileShader("shader/edge_detect.frag");
        edgeProg.link();
        edgeProg.use();
    }
    catch (GLSLProgramException& e) {
        cerr << e.what() << endl;
```

```
        exit(EXIT_FAILURE);
    }
}

void SceneBasic_Uniform::update(float t)
{
    float deltaT = t - tPrev;
    if (tPrev == 0.0f)
        deltaT = 0.0f;
    tPrev = t;
    angle += 0.25f * deltaT;
    if (angle > glm::two_pi<float>())
        angle -= glm::two_pi<float>();

    GLFWwindow* win = glfwGetCurrentContext();
    if (win)
    {
        //blade toggle
        if (glfwGetKey(win, GLFW_KEY_Q) == GLFW_PRESS &&
!Q_Pressed) {
            bladeOn = !bladeOn;
            updateFogColour();

        }
        Q_Pressed = (glfwGetKey(win, GLFW_KEY_Q) == GLFW_PRESS);
        // colour cycling
        if (glfwGetKey(win, GLFW_KEY_E) == GLFW_PRESS && !E_Pressed)
        {
            bladeColourIndex = (bladeColourIndex + 1) %
(int)bladeColours.size();

            bladeEmissive.use();
            bladeEmissive.setUniform("BladeColour",
bladeColours[bladeColourIndex]);

            updateFogColour();
        }
        E_Pressed = (glfwGetKey(win, GLFW_KEY_E) == GLFW_PRESS);
        //fog stuff
        if (glfwGetKey(win, GLFW_KEY_F) == GLFW_PRESS && !F_Pressed)
        {
            fogEnabled = !fogEnabled;
        }
        F_Pressed = (glfwGetKey(win, GLFW_KEY_F) == GLFW_PRESS);
        //rusty toggle
        if (glfwGetKey(win, GLFW_KEY_R) == GLFW_PRESS && !R_Pressed)
        {
            rusty = !rusty;
        }
        R_Pressed = (glfwGetKey(win, GLFW_KEY_R) == GLFW_PRESS);
        //rotate hilt
        if (glfwGetKey(win, GLFW_KEY_W) == GLFW_PRESS)
            hiltYawDeg -= hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_S) == GLFW_PRESS)
            hiltYawDeg += hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_UP) == GLFW_PRESS)
            hiltYawDeg -= hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_DOWN) == GLFW_PRESS)
            hiltYawDeg += hiltYawSpeed * deltaT;

        if (hiltYawDeg > 360.0f) hiltYawDeg -= 360.0f;
        if (hiltYawDeg < 0.0f)   hiltYawDeg += 360.0f;
    }
    updateCamera(deltaT);
    render();
}

void SceneBasic_Uniform::render()
{

    prog.use();

    //glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT |
GL_STENCIL_BUFFER_BIT);

    // BlinnPhong spotlight
    glm::vec4 lightPosWorld = glm::vec4(10.0f * cos(angle), 10.0f, 10.0f *
sin(angle), 1.0f);
    prog.setUniform("Spot.Position", glm::vec3(view * lightPosWorld));

    glm::vec3 targetWorld = glm::vec3(0.0f, 2.0f, 3.0f);
    glm::vec3 lightDirWorldSpot = glm::normalize(targetWorld -
glm::vec3(lightPosWorld));

    glm::vec3 lightDirView = glm::mat3(view) * lightDirWorldSpot;
    prog.setUniform("Spot.Direction", lightDirView);
```

```
    glm::vec3 lightDirWorld = glm::normalize(glm::vec3(-0.6f, -1.0f,
-0.4f));
    glm::vec3 lightDirViewSpace = glm::mat3(view) * -lightDirWorld;
    prog.setUniform("Light.Position", glm::vec4(lightDirViewSpace, 0.0f));


    prog.setUniform("FogEnabled", fogEnabled ? 1 : 0);
    prog.setUniform("FogScale", fogScale);

    // Skybox stuff
    skyboxShader.use();
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);
    skyboxShader.setUniform("SkyboxTex", 0);

    glm::mat4 skyView = glm::mat4(glm::mat3(view));
    glm::mat4 skyMVP = projection * skyView;
    skyboxShader.setUniform("MVP", skyMVP);

    skyboxShader.setUniform("FogEnabled", fogEnabled ? 1 : 0);
    skyboxShader.setUniform("FogColour", fogColour);
    skyboxShader.setUniform("SkyFogAmount", skyFogAmount);

    sky.render();




    prog.use();
    //// Workbench
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, WorkbenchDiffuseMap);

    glActiveTexture(GL_TEXTURE2);
    glBindTexture(GL_TEXTURE_2D, WorkbenchDiffuseMap);

    prog.setUniform("Material.Ks", vec3(0.0f));
    prog.setUniform("Material.Ka", vec3(0.1f));
    prog.setUniform("Material.Shininess", 180.0f);

    model = mat4(1.0f);
    model = glm::translate(model, vec3(0.0f, -1.0f, 0.0f));
    model = glm::rotate(model, glm::radians(25.0f), vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, vec3(0.5f));
    setMatrices();
    plane.render();

    // Lightsaber hilt
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, LSdiffuseTexture);

    glActiveTexture(GL_TEXTURE1);
    glBindTexture(GL_TEXTURE_2D, LSnormalMap);

    if (rusty) {
        glActiveTexture(GL_TEXTURE2);
        glBindTexture(GL_TEXTURE_2D, LSmixingTexture);
    }
    else {
        glActiveTexture(GL_TEXTURE2);
        glBindTexture(GL_TEXTURE_2D, LSdiffuseTexture);
    }

    prog.setUniform("Material.Ks", vec3(0.25f));
    prog.setUniform("Material.Ka", vec3(0.15f));
    prog.setUniform("Material.Shininess", 80.0f);
    prog.setUniform("MixAmount", 1.0f);

    model = mat4(1.0f);
    model = glm::translate(model, vec3(-1.0f, 2.0f, 2.0f));
    model = glm::scale(model, vec3(1.5f));
    model = glm::rotate(model, glm::radians(270.0f), vec3(1.0f, 0.0f, 0.0f));
    model = glm::rotate(model, glm::radians(300.0f), vec3(0.0f, 0.0f, 1.0f));
    model = glm::rotate(model, glm::radians(hiltYawDeg), vec3(0.0f, 1.0f,
0.0f));

    setMatrices();
    LightsaberMesh->render();


    if (edgeEnabled)
    {
        // ---------- HILT EDGE PASS A: render hilt mask into FBO ----------
        glBindFramebuffer(GL_FRAMEBUFFER, hiltFbo);
        glViewport(0, 0, width, height);
```

```
        glDisable(GL_BLEND);
        glEnable(GL_DEPTH_TEST);
        glDepthMask(GL_TRUE);

        glClearColor(0, 0, 0, 0);
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

        hiltMaskProg.use();

        // Reuse the same model transform you used for the hilt
        // (IMPORTANT: copy/paste the same model matrix setup)
        model = mat4(1.0f);
        model = glm::translate(model, vec3(-1.0f, 2.0f, 2.0f));
        model = glm::scale(model, vec3(1.5f));
        model = glm::rotate(model, glm::radians(270.0f), vec3(1.0f, 0.0f,
0.0f));
        model = glm::rotate(model, glm::radians(300.0f), vec3(0.0f, 0.0f,
1.0f));
        model = glm::rotate(model, glm::radians(hiltYawDeg), vec3(0.0f,
1.0f, 0.0f));

        // For the mask shader we only need MVP
        mat4 mv = view * model;
        hiltMaskProg.setUniform("MVP", projection * mv);

        LightsaberMesh->render();

        // ---------- HILT EDGE PASS B: Sobel detect over main framebuffer
----------
        glBindFramebuffer(GL_FRAMEBUFFER, 0);
        glViewport(0, 0, width, height);

        glDisable(GL_DEPTH_TEST);
        glDepthMask(GL_FALSE);

        glEnable(GL_BLEND);
        glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

        edgeProg.use();
        glActiveTexture(GL_TEXTURE0);
        glBindTexture(GL_TEXTURE_2D, hiltMaskTex);
        edgeProg.setUniform("MaskTex", 0);

        edgeProg.setUniform("TexelSize", glm::vec2(1.0f / width, 1.0f /
height));
        edgeProg.setUniform("Threshold", edgeThreshold);
        edgeProg.setUniform("Opacity", edgeOpacity);
        edgeProg.setUniform("Thickness", edgeThickness);
        edgeProg.setUniform("EdgeColour", edgeColour);

        glBindVertexArray(fsTriVao);
        glDrawArrays(GL_TRIANGLES, 0, 3);
        glBindVertexArray(0);

        // Restore defaults
        glDisable(GL_BLEND);
        glDepthMask(GL_TRUE);
        glEnable(GL_DEPTH_TEST);
    }


    // Blade
    if (bladeOn)
    {
        glm::mat4 bladeBase = mat4(1.0f);
        bladeBase = glm::translate(bladeBase, vec3(10.0f, 2.1f, -4.3f));
        bladeBase = glm::rotate(bladeBase, glm::radians(270.0f), vec3(1.0f,
0.0f, 0.0f));
        bladeBase = glm::rotate(bladeBase, glm::radians(300.0f), vec3(0.0f,
0.0f, 1.0f));

        // -------- Pass 1: White core (writes stencil = 1 where the blade is)
--------
        bladeEmissive.use();

        glDisable(GL_BLEND);
        glEnable(GL_DEPTH_TEST);
        glDepthMask(GL_TRUE);

        // Stencil: always write 1 where fragments pass depth test
        glStencilMask(0xFF);
        glStencilFunc(GL_ALWAYS, 1, 0xFF);
        glStencilOp(GL_KEEP, GL_KEEP, GL_REPLACE);

        model = bladeBase;
        model = glm::scale(model, vec3(0.4f));
```

```
        setMatricesEmiss();

        bladeEmissive.setUniform("BladeColour", glm::vec3(1.0f));
        bladeEmissive.setUniform("Intensity", 10.0f);
        bladeEmissive.setUniform("RimPower", 5.0f);
        bladeEmissive.setUniform("RimIntensity", 0.25f);

        bladeEmissive.setUniform("Alpha", 1.0f);
        bladeEmissive.setUniform("CoreAlpha", 1.0f);

        BladeMEsh->render();

        // -------- Pass 2: Coloured glow shell (only OUTSIDE the core
silhouette) --------
        glEnable(GL_BLEND);
        glBlendFunc(GL_ONE, GL_ONE);

        // Don't write depth for glow, but DO depth test so hilt blocks it
        glDepthMask(GL_FALSE);

        // Stencil: only draw where stencil != 1 (i.e., outside the core)
        glStencilMask(0x00);
        glStencilFunc(GL_NOTEQUAL, 1, 0xFF);
        glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);

        // Optional but usually improves the "wrap" look:
        // render only backfaces of the enlarged shell so it appears as an
outline
        glEnable(GL_CULL_FACE);
        glCullFace(GL_FRONT);

        model = bladeBase;
        model = glm::scale(model, vec3(0.4f * 1.4f));
        setMatricesEmiss();

        bladeEmissive.setUniform("BladeColour",
bladeColours[bladeColourIndex]);
        bladeEmissive.setUniform("Intensity", 2.5f);
        bladeEmissive.setUniform("RimPower", 2.0f);
        bladeEmissive.setUniform("RimIntensity", 1.0f);

        bladeEmissive.setUniform("Alpha", 0.55f);
        bladeEmissive.setUniform("CoreAlpha", 0.0f);

        BladeMEsh->render();

        // -------- Restore state --------
        glCullFace(GL_BACK);
        glDisable(GL_CULL_FACE);

        glDepthMask(GL_TRUE);
        glDisable(GL_BLEND);

        glStencilMask(0xFF);
        glStencilFunc(GL_ALWAYS, 0, 0xFF);
        glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);
    }
}

void SceneBasic_Uniform::setMatrices() {
    mat4 mv = view * model;
    prog.setUniform("ModelViewMatrix", mv);
    prog.setUniform("NormalMatrix", glm::mat3(vec3(mv[0]), vec3(mv[1]),
vec3(mv[2])));
    prog.setUniform("MVP", projection * mv);
}

void SceneBasic_Uniform::setMatricesEmiss() {
    mat4 mv = view * model;
    bladeEmissive.setUniform("ModelViewMatrix", mv);
    bladeEmissive.setUniform("NormalMatrix", glm::mat3(vec3(mv[0]),
vec3(mv[1]), vec3(mv[2])));
    bladeEmissive.setUniform("MVP", projection * mv);
}

void SceneBasic_Uniform::updateFogColour()
{
    fogColour = bladeOn ? bladeColours[bladeColourIndex] : fogGrey;

    // Update main scene shader fog colour
    prog.use();
    prog.setUniform("Fog.Colour", fogColour);

    // Update skybox fog colour too (so it stays consistent)
    skyboxShader.use();
    skyboxShader.setUniform("FogColour", fogColour);
}
```

```
void SceneBasic_Uniform::resize(int w, int h) {
    glViewport(0, 0, w, h);
    width = w;
    height = h;
    projection = glm::perspective(glm::radians(70.0f), (float)w / h, 0.3f,
100.0f);
    initHiltEdgeFbo(w, h);
}

static float wrapDeg(float a)
{
    while (a > 180.0f) a -= 360.0f;
    while (a < -180.0f) a += 360.0f;
    return a;
}

void SceneBasic_Uniform::updateCamera(float dt)
{
    GLFWwindow* win = glfwGetCurrentContext();
    if (!win) return;

    // Orbit with A/D (and Left/Right)
    float yawInput = 0.0f;
    if (glfwGetKey(win, GLFW_KEY_A) == GLFW_PRESS || glfwGetKey(win,
GLFW_KEY_LEFT) == GLFW_PRESS)
        yawInput -= 1.0f;
    if (glfwGetKey(win, GLFW_KEY_D) == GLFW_PRESS || glfwGetKey(win,
GLFW_KEY_RIGHT) == GLFW_PRESS)
        yawInput += 1.0f;

    camYawDeg += yawInput * camYawSpeedDeg * dt;

    if (autoRecentre)
    {
        if (yawInput == 0.0f)
            camYawDeg = glm::mix(camYawDeg, yawCentreDeg, 1.0f -
expf(-yawReturnSpeed * dt));

        camPitchDeg = glm::mix(camPitchDeg, pitchCentreDeg, 1.0f -
expf(-pitchReturnSpeed * dt));
    }

    // Mouse orbit (only while cursor is captured)
    if (cursorCaptured)
    {
        double mx, my;
        glfwGetCursorPos(win, &mx, &my);

        if (firstMouse)
        {
            lastMouseX = mx;
            lastMouseY = my;
            firstMouse = false;
        }

        double dx = mx - lastMouseX;
        double dy = my - lastMouseY;
        lastMouseX = mx;
        lastMouseY = my;

        camYawDeg += (float)dx * mouseSensitivity;
        camPitchDeg -= (float)dy * mouseSensitivity; // move mouse up -
> pitch up

        // Vertical restriction
        camPitchDeg = glm::clamp(camPitchDeg, pitchMinDeg,
pitchMaxDeg);

        // Optional horizontal restriction
        if (limitYaw)
        {
            float rel = wrapDeg(camYawDeg - yawCentreDeg);
            rel = glm::clamp(rel, -yawLimitDeg, yawLimitDeg);
            camYawDeg = yawCentreDeg + rel;
        }
```

ChatGPT 5.2 Thinking

Share

```
    // Build orbit position from yaw, pitch, distance
    float yawRad = glm::radians(camYawDeg);
    float pitchRad = glm::radians(camPitchDeg);

    glm::vec3 offset;
    offset.x = camDistance * cosf(pitchRad) * cosf(yawRad);
    offset.y = camDistance * sinf(pitchRad);
    offset.z = camDistance * cosf(pitchRad) * sinf(yawRad);
```

```
    camPos = camTarget + offset;

    // Always keep focus on the target (hard lock)
    view = glm::lookAt(camPos, camTarget, camUp);
}

void SceneBasic_Uniform::initHiltEdgeFbo(int w, int h)
{
    // Clean up old
    if (hiltMaskTex) glDeleteTextures(1, &hiltMaskTex);
    if (hiltDepthRb) glDeleteRenderbuffers(1, &hiltDepthRb);
    if (hiltFbo) glDeleteFramebuffers(1, &hiltFbo);

    // Colour mask texture
    glGenTextures(1, &hiltMaskTex);
    glBindTexture(GL_TEXTURE_2D, hiltMaskTex);
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA8, w, h, 0, GL_RGBA,
GL_UNSIGNED_BYTE, nullptr);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
GL_CLAMP_TO_EDGE);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
GL_CLAMP_TO_EDGE);

    // Depth buffer
    glGenRenderbuffers(1, &hiltDepthRb);
    glBindRenderbuffer(GL_RENDERBUFFER, hiltDepthRb);
    glRenderbufferStorage(GL_RENDERBUFFER,
GL_DEPTH_COMPONENT24, w, h);

    // FBO
    glGenFramebuffers(1, &hiltFbo);
    glBindFramebuffer(GL_FRAMEBUFFER, hiltFbo);
    glFramebufferTexture2D(GL_FRAMEBUFFER,
GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, hiltMaskTex, 0);
    glFramebufferRenderbuffer(GL_FRAMEBUFFER,
GL_DEPTH_ATTACHMENT, GL_RENDERBUFFER, hiltDepthRb);

    GLenum drawBufs[] = { GL_COLOR_ATTACHMENT0 };
    glDrawBuffers(1, drawBufs);

    GLenum status = glCheckFramebufferStatus(GL_FRAMEBUFFER);
    if (status != GL_FRAMEBUFFER_COMPLETE) {
        std::cerr << "Hilt edge FBO incomplete: " << status << std::endl;
    }

    glBindFramebuffer(GL_FRAMEBUFFER, 0);
}
```

ive tweaked a couple of the variables but it now feels awkward and doesnt pan around the model as i was hoping, can you help me tweak things to resolve this

Thought for 28s ›

Yep — I can see exactly why it feels awkward, and there are **two big structural issues** in what you've pasted that will *guarantee* "wonky" orbit behaviour.

0) First: your header still contains the old FPS camera bits (and one is actually poisonous)

This line is still active:

```
C++
```

...but **at that point in the class,** `camPos` **hasn't been declared yet** (you commented out the earlier `camPos` and then declared a new one later). That alone can cause compile issues or undefined behaviour depending on what you changed around it.

Fix: delete (or fully comment out) the entire old "Camera stuff" block. For orbit, you do not want `camFront` at all.

So in `.h`, remove this whole bit:

```
// Camera stuff
///glm::vec3 camPos = ...
C++
```

Keep only the orbit block.

1) Your "autoRecentre" is fighting your clamps (and it's set to an impossible pitch)

Right now:

- `pitchMinDeg = 15`, `pitchMaxDeg = 55`
- but `pitchCentreDeg = -20` ❌

So every frame, autoRecentre tries to drag pitch toward **-20°**, then you clamp it back to **15°** when the mouse runs. That creates a constant "sticky / springy / awkward" feel.

Fix: set `pitchCentreDeg` to something *inside your clamp range*, ideally your starting pitch.

---

2) You're forcing the camera to start at a pitch that puts it under the model

In `initScene()` you still do:

```
C++
```

That's the original cause of the camera dropping under the plane. You already have the correct starting `camPos`, so don't guess angles — **derive them**.

The clean fix (do these 3 changes)

A) In `initScene()` derive yaw/pitch from your original camera position

Replace your "preferred starting angles" section with this:

```
// Derive orbit angles from your current placement (so it starts EXACTLY where it used to)
glm::vec3 off = camPos - camTarget;
camDistance = glm::length(off);

glm::vec3 dir = off / camDistance;

camYawDeg   = glm::degrees(atan2f(dir.z, dir.x)); // [-180..180]
camPitchDeg = glm::degrees(asinf(dir.y));         // [-90..90]

// Set centre + limits around that starting pitch (keeps height sensible)
yawCentreDeg   = camYawDeg;
pitchCentreDeg = camPitchDeg;

C++

pitchMaxDeg = camPitchDeg + 15.0f;   // allow a bit higher than start
```

Then keep:

```
C++
```

This alone will make it start correctly and "orbit around the model" the way you expect.

B) Move autoRecentre to the end, and only apply it when you're not moving the mouse

Right now you apply autoRecentre **before** mouse input, which makes the camera feel like it's sliding around underneath you.

Change your `updateCamera()` flow to:

1. read keyboard input
2. read mouse input (and track whether mouse moved)
3. clamp pitch
4. apply autoRecentre **only if no inputs happened**

Use this edit:

```
void SceneBasic_Uniform::updateCamera(float dt)
{
    GLFWwindow* win = glfwGetCurrentContext();
    if (!win) return;

    float yawInput = 0.0f;
    if (glfwGetKey(win, GLFW_KEY_A) == GLFW_PRESS || glfwGetKey(win, GLFW_KEY_LEFT) == GLFW_PRESS
    if (glfwGetKey(win, GLFW_KEY_D) == GLFW_PRESS || glfwGetKey(win, GLFW_KEY_RIGHT) == GLFW_PRES

    camYawDeg += yawInput * camYawSpeedDeg * dt;

    bool mouseMoved = false;

    if (cursorCaptured)
    {
```

```
        double mx, my;
        glfwGetCursorPos(win, &mx, &my);

        if (firstMouse)
        {
            lastMouseX = mx;
            lastMouseY = my;
            firstMouse = false;
        }

        double dx = mx - lastMouseX;
        double dy = my - lastMouseY;
        lastMouseX = mx;
        lastMouseY = my;

        mouseMoved = (dx != 0.0 || dy != 0.0);

        camYawDeg   += (float)dx * mouseSensitivity;
        camPitchDeg -= (float)dy * mouseSensitivity;
    }

    // Clamp pitch AFTER all changes
    camPitchDeg = glm::clamp(camPitchDeg, pitchMinDeg, pitchMaxDeg);

    // Optional yaw restriction
    if (limitYaw)
    {
        float rel = wrapDeg(camYawDeg - yawCentreDeg);
        rel = glm::clamp(rel, -yawLimitDeg, yawLimitDeg);
        camYawDeg = yawCentreDeg + rel;
    }

    // Auto recentre ONLY when no input
    if (autoRecentre && yawInput == 0.0f && !mouseMoved)
    {
```

C++

```
    {

    float yawRad   = glm::radians(camYawDeg);
    float pitchRad = glm::radians(camPitchDeg);

    glm::vec3 offset;
    offset.x = camDistance * cosf(pitchRad) * cosf(yawRad);
    offset.y = camDistance * sinf(pitchRad);
    offset.z = camDistance * cosf(pitchRad) * sinf(yawRad);

    camPos = camTarget + offset;
    view = glm::lookAt(camPos, camTarget, camUp);
}
```

This makes it feel *much* less "floaty".

C) Tweak values so it feels like a controlled orbit (good defaults)

Start with these:

C++

```
float camYawSpeedDeg = 75.0f;      // slightly slower orbit
float mouseSensitivity = 0.06f;    // less twitchy

bool autoRecentre = false;         // turn OFF while tuning (seriously)
```

Once the orbit feels good, re-enable autoRecentre but keep it subtle:

C++

```
autoRecentre = true;
yawReturnSpeed = 1.5f;
pitchReturnSpeed = 1.5f;
```

Also, if you want "rubber band but not full recentre", don't use autoRecentre — just use yaw/pitch limits:

C++

```
limitYaw = true;
yawLimitDeg = 25.0f; // tight horizontal window around centre
```

Quick sanity check: why it "doesn't pan around the model"

The biggest reason in your current paste is:

- you're still **forcing** `camYawDeg = -45`, `camPitchDeg = -20` in `initScene()` even though you changed header defaults.
- autoRecentre pitch is **-20** while your allowed pitch range is **15..55**
- autoRecentre runs regardless of input ordering, so it's constantly yanking the camera

Fixing those gives you the orbit you expected.

If you want, tell me **what point you want to orbit around** (the saber hilt isn't at `(0, 0.75, 0)` in your scene — it's around `(-1, 2, 2)`), and I'll give you the exact `camTarget` to match your actual "main model" so the

orbit is centred perfectly.